Page No.
Date

Revision <ML>

① df = pd.read_csv(r'filepath')

② df.head() ⇒ print first 5 rows.

③ df.shape ⇒ (row, colow)

④ df.dtypes ⇒ Shows data types of each columns.

⑤ df.select_dtypes(include = ['int', 'float']) ⇒ numerical columns

⑥ df.select_dtypes(include = ['category']) ⇒ categorical columns.
get of Hered out.

⑦ df.isnull().Sum() = gives total no of null values in each columns.
col1 - 0
col2 - 1
col3 - 2.

⑧ df.duplicated().Sum() ⇒ total no of duplicate rows.

Drop rows contain NULL

df = df.dropna(Subset = ['Satisfaction', 'columnName2' ...])

⑨ df = df.duplicates()

df = df.drop_duplicates() ⇒ If duplicate exists then remove it.

⑩ df.describe() ⇒ Describes all statistical parameters.
column columnName ← features.

- count
- mean
- std.
- min.
- 25%
- 50%
- 75%
- max

(11) ~~df.column [~~

df[columnName][boolean_cond^n]

eg:

	Name	Age	Marks
0	A	18	80
1	B	25	90
2	C	30	70
3	D	15	60

df['Age'] > 20

0	False
1	True
2	True
3	False

df [df['Age'] > 20]

Only rows where age > 20

	Name	Age	Marks
0	B	25	90
1	C	30	70

Page No.
Date

(12) len(df) = how many rows are total in the data frame.

(13) df.isnull() ⇒ returns a table with only boolean values.

(14) df[columnName].fillna(value) =

applies on a particular column and replaces all its Null values with the 'value'.

(15) df[columnName].mode()[0] = applies on the particular column and finds it mode value.

(16) df[columnName].fillna(df[columnName].mode()[0]) ⇒ applies or replace all null value with the mode value of column.

Note: mode may give a Series so index zero[0]

(15) df[columnName].unique() = returns only the unique values present in the column.

~~array([~~ eg: df['city'].unique() →

Output ⇒ array(['Mumbai', 'Delhi', 'Kolkata'], dtype = object)

(16) df['columnName'].value_counts() → It is used to count how many times each unique value appears in the column.

eg: df['city'].value_counts()

City.
Mumbai 2.

dtype: int64

Page No.
Date

Python Series is a one dimension labeled array that store data of any type (numbers, strings, objects etc).

Think of a Series as a single column of an excel sheet with row labels (indexes).

eg- df['city'].value_counts().

city
Mumbai 2
Delhi 3
Raipur 4
Name: count int64

∴ The output gives a Series named as 'count' having (integer) type of data.

(12) ▶ Converting Review = into number:

```
def Convert_Size (variable):
    if variable.endswith('M'):
        return float (variable[:-1]) * 1000
```

Slicing.

∴

① S12 K [:-1] = S12.

and float(S12.0) * 1000

returns S12000.

Page No.
Date

Scaling.

(18)

```
from sklearn.preprocessing import StandardScaler.
scaler = StandardScaler()
num_cols = ["Age", "Fare", "SubSip", "Parch"]
df[num_cols] = scaler.fit_transform(df[num_cols]).
```

Standard feature scaling technique that transforms numerical features so that they have:

- Mean = 0.
- Standard deviation = 1.

It is also called Z-score normalization.

Why need for Scaling :-

ML algorithm calculates weights, distance or gradients. If one feature has a much large range than another, it can dominate the model.

ex :-

Age	Salary
20	30000
30	40000
40	50000

How it works :- $z = \frac{x - \mu}{\sigma}$

x = original value

μ = mean of the feature

σ = Standard deviation

z = scaled value.

Suppose $\mu = \frac{20 + 40 + 30}{3} = 30$

$\sigma \simeq 8.16$

	Scale Age	Salary
① Values less than mean = -ve	-1.22	
② Value = mean ⇒ 0	0	
③ Value > mean = +ve	1.22	

∴ after scaling :-

Page No.
Date

Which algorithms needs Scaling

Need	Not need
① Logistic Regression. ✓	
② Linear Regression. ✓	Decision Tree ✗
③ SVM. ✓	Random Forest ✗
④ KNN. ✓	
⑤ K-means clustering. ✓	Gradient Boosting. ✗
⑥ Neural Networks. ✓	
⑦ PCA.	XG Boost. ✗

Standard Scaler vs MinMaxScaler.

mean = 0 values as 0 and 1.

Uses mean & standard deviation

ML model

Uses minimum and maximum.

Neural network.

⑩ Data cleaning:

① Removal of null values.

② Removal of duplicate value.

③ Encoding.

④ Conversion of categorical data to numerical data.

Page No.
Date

2) One hot encoding.

It is a technique used to convert categorical data (text labels) into numerical values so that machine learning algorithms can understand them.

after encoding

Person	Gender	Gender_Male	Gender_Female
A	Male	1	0
B	Female	0	1
C	Male	1	0

df = pd.get_dummies(df, column = ['columnName'])

instead of assigning no = Male = 0 and female = 1, machine may assume female > male.

Dataset Splitting [80:20]

```
from sklearn.model_selection import
    train_test_split
x = df.drop ("Survived", axis = 1)
y = df ["Survived"]  #target.

x_train, x_test, y_train, y_test = train_test_split
                    (x, y, test_size = 0.20,
                    random_state = 42)
```

Page No.
Date

(22) MSE and R2 score.

```
from sklearn.linear_model import LinearRegression.
from sklearn.metrics import mean_squared , r2_score.
```

MSE (Mean Squared Error) = $\frac{1}{n}\Sigma(\text{Actual} - \text{Predicted})^2$.

smaller MSE = better model.

R2-Score : measures how well the model explains the data.

1.0 = Perfect. 0.5 = Average.

0.9 = very good.

(23) Linear Regression {Regression tasks}.

```
model = LinearRegression()
model.fit (X_train , Y_train)
```

after that model can map input feature with target feature.

$y = mx + c$.

▼ m: coefficient:

```
print ("coefficient" , model.coef[0])
```

Predict new value ➤

```
model.predict (newValue)
```

(24) Matplotlib & Numpy

Matplotlib is a widely used python library for creating static, animated, and interactive visualizations.

instead of manual array

x = [1, 2, 3, 4, 5]
y = [1, 4, 9, 16, 25]

→ point = np.array([1, 6])
1, 2, 3, 4, 5, 6.

plt.plot(x, y)
plt.xlabel("xvalues")
plt.ylabel("Ylabel")
plt.title("Simple Line plot")
plt.show()

■ Common plot types :-

Line plot :
plt.plot(x, y)

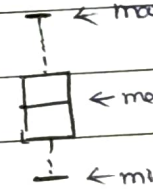
Scatter plot :
plt.scatter(x, y)

Bar chart :
plt.bar(x, y)

Histogram :
plt.hist(y)

Pie chart :
plt.pie([20, 30, 50], labels = ["A", "B", "C"])

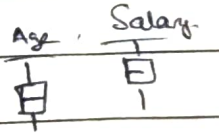
Box plot : (with Pandas very popular, but matplotlib also has option)

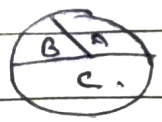


df = pd.read_csv("data.csv")
df.boxplot()
plt.show()

Eg :-

	Age	Salary
0	20	30,000
1	25	40,000
2	30	50,000
3	35	1,00,000





Page No.
Date

Customizing a plot :-

① plt.plot (x, y, marker = "o", linestyle = "--").

marker defines the shape of a datapoint :-

"o" → circle.

'x' → cross.

's' → square.

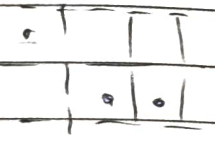
linestyle → how the line looks connecting the points.

② plt.plot (x, y, linewidth = value).

controls the boldness or width of the line.

~~③ plt.xlabel ("Time") :-~~

④ plt.grid (True). ≃ adds background reference line.



⑤ X axis limit :- plt.xlim (0, 10) ⇒ shows only x value for 0 to 10.

⑥ Y axis limit :- plt.ylim (0, 100) → shows only Y values from "0 to 100".

Page No.
Date

2) Changing column Name :-

column_name = ["x", "y", "z"]
df = pd.read_csv ("%.csv", name = columnNames)

2) Unique total of categories in a column.

~~df['Target'].value_count()~~
eg :- df ['Gender'].value_count().

Gender
Male 200
Female 200

2) Replace specific values in Data frame with new values :-

df.replace (old_value, new_value).

> Replace a single :-

import pandas as pd.
df = pd.Dataframe ({ 'Status' : ["Yes", "No", "Yes", "No"] }).
print (df).

df.replace ("Yes" : 1)

	Status
0	1
1	NO
2	1
3	No

df.replace ({
"Yes" : 1,
"No" : 0
}).

	Status
0	1
1	0
2	1
3.	0.

~~df~~ →

		Status
①	0	Yes.
	1	No.
	2	Yes.
	3	NO.

Page No.
Date

Stemming / cleaning :-

this Stemming is a technique in Natural Language Processing (NLP) that reduces words to their root / base by removing prefixes or suffixes.

def Stemming (content):

Stem_content = re.sub ('[^a-zA-Z]', " ", content)

replace everything which is not a letter.

eg :- I am 123 !!

↳ I am .

Stem_content = Stem_content.lower(),

converts everything to lower case.

Stem_content = Stem_content.split()

converts a sentence into a list of word wise

.split()

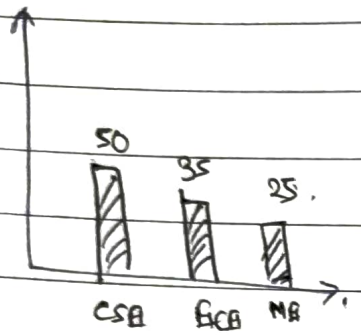
['i'

'love',

'python'].

~~Assis~~

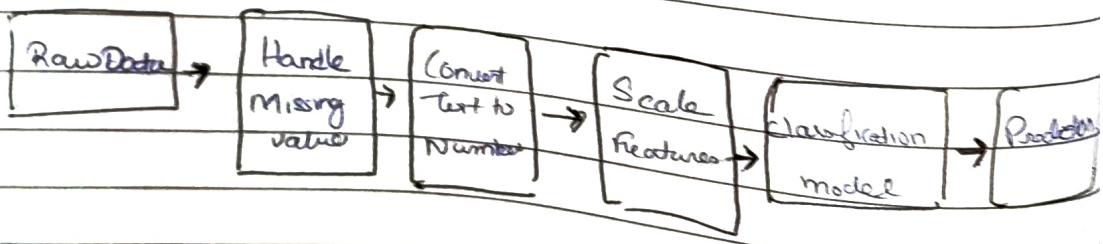
(28) df['department'].value_counts().plot(kind="bar")



* axis (horizontal) → categories.

(29) **Classification Pipeline :-**

A pipeline is simply a sequence of steps that data follows before reaching the actual model and start of learning.



This pipeline ensures the same preprocessing is applied everytime.

① Imputation :- The process of replacing missing value is called imputation

② Ordinal encoding :

The necessary encoding

③ Standard Scaling :

Machine may get dominated by some columns like Salary, so we need to scale value.

for Scaled_value = (x - mean) / Standard deviation.

eg :- df = Pd.read_csv ("data.csv")

Age	Salary	Gender	Purchased
25	50000	Male	1
NaN	60,000	Female	0
30	NaN	Male	1

- Purchased = target column
- Age, Salary = Numerical data
- Gender = Categorical data

★ code of Pipeline :

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer

num_pipeline = Pipeline ([
    ("imputer",
     SimpleImputer (Strategy = "mean")),
    ("scaler", StandardScaler ())])
```

Page No.
Date

Input Data → Imputer → Scaler → Output Data

- **Syntax**-

```
Pipeline ([
    ("StepName 1", transformer_1()),
    ("Step Name 2", transformer_2()),
    ...
]).
```

A Pipeline expects a list of tuple

① SimpleImputer (Strategy = "mean").
" " = "median".
" " = "most_frequent"
" " = "constant"

② ("encoder", OneHotEncoder (handle_unknown = "ignore"))

Suppose during training [Gender = [male, female]] but is encoded but during testing [Gender = [male, female, other]] came then will give error.
So to handle error we use "ignore".

Page No.
Date

(20) Numerical & Categorical pipeline :-

① Fill Missing values ② Standard Scaling :

```
num_pipeline = Pipeline ([
    ("imputer_Adak", SimpleImputer (strategy = "mean")),
    ("Scaler_Adak", StandardScaler ())
])
```

① fill missing values ② One hot encoding :

```
categorical_pipeline = Pipeline ([
    ("imputer_Adak", SimpleImputer (strategy = 'most_frequent')),
    ("encod_Adak", OneHotEncoding ( handle_unknown = "ignore"))
])
```

(21) Now extract numerical columns from data frame.

```
numerical = ["Age", "Salary"]   [← no need of df.]
categorical = ["Gender"]
```

(22) Now connecting Columns with pipeline :-

```
preprocessor = ColumnTransformer ([
    ("num", num_pipeline, numerical_columns),
    ("category_Score", Category_pipeline, categorical_columns)
])
```

column name

Page No.
Date

33 Model Comparison :-

① Logistic Regression = It is a classification algorithm.

It predicts probability :-

If $p(x) > 0.5$ → goto class 1.

else goto class 0.

② Random Forest :- { evaluate multiple decision tree and accordingly make decisions.

Decision Tree 1 → Yes.

Decision Tree 2 → Yes.

" 3 → No.

" 4 → Yes.

∴ Majority vote → Yes.

③ XGboost :- Extreme boost.

Accuracy :-

Logistic Regression < Decision Tree < Random Forest < XGboost.

24) ● Confusion Matrix :

Suppose 1 → Diseases
0 → Healthy

	Predicted 1	Predicted 0
Actual 1	True Positive	False Negative
Actual 0	False Positive	True Negative

code :

```
from sklearn.metrics import RocCurveDisplay, from_estimation (model, X_test, y_test),
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
# Generate confusion matrix
cm = confusion_matrix (y_test, y_pred)
# Print confusion matrix
print(cm)
# Display confusion matrix
disp =
```

(35) Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

(36) $$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall (Sensitivity)} = \frac{TP}{TP + FN} \quad \left[\text{Recall} = \frac{TP}{(TPR)} \quad TP + FN\right]$$

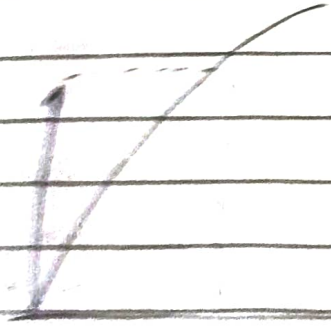
(37) F1 Score

$$2\pi$$

- ROC Curve

Receiver Operating characteristics.

True positive rate / Recall = (TPR) vs False Positive Rate (FPR)



code :-

from sklearn.metrics import RocCurveDisplay

RocCurveDisplay.from_estimator(model, X_test, y_test)

(36)

confusion matrix
Accuracy
Precision → (Y_test, Y_pred)
Recall
F1-Score
R^2-Score

ROC curve → (model, X_test, Y_test)

(37) Under fitting & Over fitting condition.

underfitting ⇒ [Training R^2 << Testing R^2]

overfitting [Training R^2 >> Testing R^2).

(38) Correlation Matrix :-

A correlation matrix shows the relationship between numerical features in a dataset.